Monadic Types and the `foreach` Comprehension in f1r3|@ng

A synchronous comprehension over declared data monads, with explicit cost

L.G. Meredith
F1R3FLY.io

October 2026 — draft

Abstract

f1r3|@ng (spoken *f1r3lang*; formerly MeTTaIL, and before that rholang 1.4) has one comprehension, `for`, and it binds in the *space*: its sources are channels, its data are shared with concurrent writers, and when several candidate matchings are available it resolves them by choosing one. This note adds a second comprehension, `foreach`, which binds in a *declared data monad*: its sources are terms of a monadic sort, it meets no writer, and when a nondeterministic monad presents several candidates it resolves them by summing over all of them. The two comprehensions share a surface and a vocabulary — patterns, per-group `where` clauses, `;` for sequence — and differ in exactly the place where binding happens.

The design has three parts. In the data definition sublanguage, a `Monads` builder declares that a sort is a monad and gives the interpretation of unit, flatMap and optionally map as rewrites, together with four optional capabilities — a zero, commutativity, a measure, and an algebra on processes — each of which licenses a specific feature of the comprehension. The monad laws are recorded as proof obligations, not as equations, so that matching modulo a theory never meets higher-order unification. In the control flow sublanguage, `foreach` is synchronous in a precise sense: its binding skeleton is a burst of internal steps that no other process can observe, so it may be treated as a single step and sequenced with `;` exactly as the synchronous send of rholang 1.1 is. In the cost-accounted surface, the comprehension carries two gates, `{% foreach( p <- m ) yield {% P %}[t] %}[s]`, the outer funding every *binding* and the inner funding every *body*. Per-binding charging turns the data-dependent cost of the comprehension into an estimation problem: a linear funding proof needs a bound on the measure of the monadic instance, and the note sets out where such bounds come from.

Because `foreach` is a conservative extension of the core, it need not be implemented by translation into `for`. The translation is retained as the reference semantics, and a native implementation is correct when it is *gradedly* bisimilar to it — the same behaviour and the same bill.

Contents

1 Two comprehensions **2**

2 Background **4**
2.1 The data definition sublanguage 4
2.2 The control flow sublanguage 4
2.3 The cost-accounted surface 5

2.4 Monads one level up . . . 5

3 Declaring a data monad **5**
3.1 The prerequisite: a sort constructor . . . 5
3.2 The `Monads` builder . . . 6
3.3 Monads and the algebra of theories . . . 7

4 The `foreach` comprehension **8**
4.1 Syntax . . . 8
4.2 Static discipline . . . 8
4.3 Groups: `;` and `&` . . . 9
4.4 `where` in `foreach` . . . 9
4.5 `yield`, and the algebra on processes . . . 9
4.6 Reference semantics . . . 9
4.7 Synchrony . . . 10

5 Sequencing **11**

6 Cost **12**
6.1 Two gates . . . 12
6.2 The atomic outer gate . . . 13
6.3 Precharge and linear funding proofs . . . 13

7 Implementation **14**
7.1 The translation is the specification, not the strategy . . . 14
7.2 What native evaluation reuses . . . 15
7.3 What native evaluation must not inherit . . . 15
7.4 What changes in the representation . . . 15
7.5 What gets simpler . . . 16
7.6 Built-in and declared monads . . . 16
7.7 Where the frontend stands . . . 16

8 Interactions with the rest of f1r3|@ng **16**
8.1 Channel types . . . 16
8.2 Generated logics . . . 17
8.3 Graded `where` clauses . . . 17
8.4 Simulation . . . 17
8.5 The quantum fence . . . 17
8.6 Summary . . . 18

9 A first tranche **18**

10 Open questions **18**

1 Two comprehensions

The comprehension is the shape data access takes when written down honestly: name a source, filter it, project the survivors, act. SQL, LINQ [5], XQuery's FLWOR and the list comprehension [7] are the same shape, and Wadler's observation that the comprehension is monadic [8], in the sense

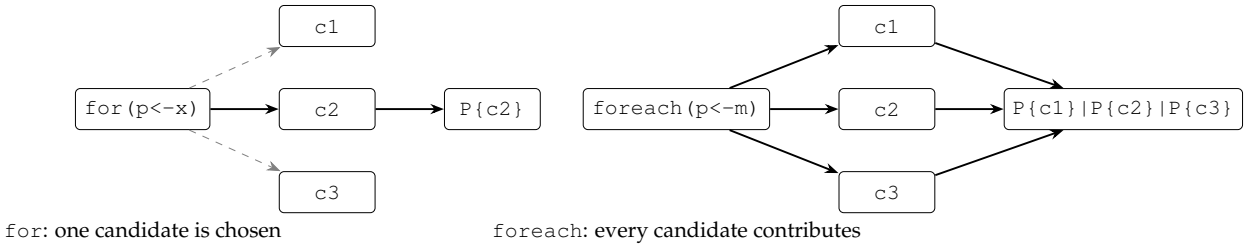


Figure 1: The two resolution disciplines for candidates. Over a bag, the aggregate on the right is literally a parallel composition, because `PPar` is a bag (§4.5).

Moggi gave to notions of computation [6], is what makes the family a family: every one of them is a notation for flatMap in some monad.

f1r3|@ng's receipt

```
for( ptrn <- chan where cond ) P
```

is that shape with the `DO` made first-class [11]. But the monad it binds in is unusual. The source is a channel; the datum is produced by a concurrently running writer; the join `&` is a rendezvous; the communication rule substitutes the datum into the continuation and thereby *witnesses* the meeting. Call this the *space* monad. Its bind is asynchronous, it is shared, and it is nondeterministic in the strong sense that competing readers race.

Developers also need the other kind of comprehension — the one LINQ and Scala's `for` provide over a collection already in hand. A term received on a channel may itself be a list, a bag, a tree of findings, a distribution. Walking it with `for` is possible only by first spilling it onto a channel, which is both expensive and wrong: it exposes a private traversal to every other reader of that channel, and it turns a deterministic computation over a value into a race.

`foreach` is the missing comprehension. It binds in a *data monad* declared in the data definition sublanguage (DDL), and it is synchronous: the traversal is invisible from outside, so the comprehension may be treated as one step.

Choose versus sum. The cleanest way to see the difference is through the word the developer document uses for the thread running through f1r3|@ng's semantics: *candidate* [11]. A candidate is one way of assigning data to the patterns of a comprehension.

- In `for`, candidates are resolved by *choice*. One fires; the others do not.
- In `foreach` over a nondeterministic monad, candidates are resolved by *aggregation*. All of them fire, and flatMap sums their contributions.

Figure 1 draws the distinction. Section 8.3 shows that it is exactly the distinction the graded `where` clause needs.

Contributions.

1. A `Monads` builder for the DDL, with the laws as obligations and four capabilities (zero, commutativity, measure, algebra on processes) each tied to one feature of the comprehension (§3).
2. The `foreach` comprehension: its syntax, static discipline, reference semantics and a precise criterion for synchrony (§4).
3. Sequencing with `;`, extending rholang 1.1's synchronous send sugar, together with a sequenceable fragment of bodies that recovers a genuine completion barrier where one is

meaningful (§5).

4. An explicit cost-accounted surface with two gates, per-binding charging, an atomic n-ary outer gate, and a precharge discipline for linear funding proofs (§6).
5. An implementation stance — native evaluation, the translation retained as specification, graded bisimulation as the correctness criterion — and a map of what changes in the node (§7).
6. An account of the interaction with every other feature of f1r3|@ng (§8), and a first tranche (§9).

2 Background

This section fixes the parts of f1r3|@ng the feature touches. Fuller treatments are in the developer document [11] and the GSLT reference [13].

f1r3|@ng is built on the reflective higher-order rho calculus [18], presented in its own DDL as one theory among others.

2.1 The data definition sublanguage

A language definition is a `Theory`. Its body is an ordered chain of postfix builders — `Types`, `Exports`, `Replacements`, `Terms`, `Equations`, `Rewrites` — applied to a starting theory, and theories combine by join, meet and difference over shared ancestors [19]. Every element introduced by a builder carries an origin token, and two elements are identified exactly when their origins agree; this is what makes a join a pushout rather than a union, and what produces the `[join-collision]` and `[unshared-name]` diagnostics.

Term constructors are written in judgement form,

```
PInput . n:Name, ^x.p:[Name -> Proc] |- n "?" x "." "{" p "}" : Proc;
PPar . ps:HashBag(Proc) |- "{" ps.*sep("|") "}" : Proc;
```

with binders `^x.p` as first-class arguments, and three built-in collection sorts `HashBag`, `Set` and `List` over a category. There are no user-declared sort constructors: in the reference elaborator a sort is either a category or a built-in collection of one. That fact governs the first tranche of this design.

2.2 The control flow sublanguage

The comprehension has the form

```
for( ptrn_11 <- chan_11 & ... & ptrn_1n <- chan_1n where cond_1 ;
    ... ;
    ptrn_k1 <- chan_k1 & ... & ptrn_km <- chan_km where cond_k ) P
```

with `&` for simultaneity within a group, `;` for sequence between groups, and a `where` per group that may mention any variable bound in that group or an earlier one. Three arrows distinguish a consuming read `<-`, a persistent read `<=` and a peek `«-`. A failed guard leaves the datum in the space: a failed transaction is not rolled back, it never began [11].

Rholang 1.1 also provides a synchronous send, `x!?(Q); K`, which the node's normaliser desugars as

```
new r in { x!(*r, Q) | for( _ <- r ){ K } }
```

so that `K` runs once the receiver on `x` acknowledges on the fresh name `r` [20]. Chains `x1!?(Q1); ...; xn!?(Qn); K` are nested instances.

2.3 The cost-accounted surface

Cost accounting is written in the language rather than bolted onto an interpreter [9]. A signed term $\{P\}_s$, concretely `{% P %}[s]`, runs `P` once a token for signature `s` is available; a budget is a stack `s :: s :: ... :: ()` whose depth is the number of operations it funds. Signatures are ground names, sections `# P`, and compounds. The surface already admits per-clause signed binds,

```
{% for( {% y1 <- x1 %}[s1] & {% y2 <- x2 %}[s2] ) P %}[ s0 ]
```

in which each rendezvous surface is funded independently, and in the native four-sort grammar the continuation slots of `for` and send have the signed-term sort $\mathbb{T}$. That sorting — *wrapping by construction* — is what makes subject reduction coincide with the absence of unmetered execution [10].

2.4 Monads one level up

The GSLT reference treats three monads: the cost monad C, the history monad H and the Hypercube construction Hyp [13, 10, 14, 16]. All three are endofunctors on the category of theories: they take a theory and return a metered, logged or typed theory.

The monads of this note live one level down. A declaration "M is a monad" is a statement *inside* a theory, about one of its sorts. The distinction matters in two places. In prose, the phrase "the monad" becomes ambiguous; this note says *data monad* for the new kind and *theory-level monad* for C, H and Hyp. And in semantics, the theory-level monads act on theories that contain data monads, which raises the question of whether the declaration survives them.

Proposition 2.1 (Laws modulo erasure)**.** *Let theory T declare a data monad* M *whose laws hold in T. Then $\mathsf{C}(T)$ and $\mathsf{H}(T)$ carry the declaration, and the laws hold in each up to the corresponding erasure back to T, but not in general on the nose.*

Sketch. The two sides of the left unit law, $\mathsf{flatMap}(\mathsf{unit}(a), k)$ and $k\,a$, differ by one application of the $\mathsf{flatMap}$ rewrite. In $\mathsf{C}(T)$ that application is gated and consumes a token; in $\mathsf{H}(T)$ it is logged as an event. The two sides therefore differ in cost and in history, and agree after the erasures $\mathsf{C}(T) \to T$ and $\mathsf{H}(T) \to T$ studied in [15]. The right unit and associativity laws are similar. □

This is the expected behaviour, not a defect, and it anticipates the correctness criterion of §7: once cost is explicit, "the same computation" means the same behaviour *and* the same bill.

3 Declaring a data monad

3.1 The prerequisite: a sort constructor

A monad is a sort constructor $\mathsf{M}(-)$ with structure. `map` alone needs two element sorts, $(A \to B) \to \mathsf{M}(A) \to \mathsf{M}(B)$. The DDL has no user-declared sort constructors, so the design proceeds in two steps.

Step 1: the built-in collections. `List`, `Set` and `HashBag` are already sort constructors, and they are the canonical monads: the list monad, the finite powerset monad, and the finite multiset monad. Declaring them monadic needs no new sort machinery. They are declared once, in a prelude supplied by the toolchain, and are available to every theory.

Step 2: sort parameters. The general feature extends `Types` with parameterised categories,

```
Types { Tree(A); }
```

where `A` ranges over categories. This is the real feature, and it is independently useful: the Hypercube construction currently generates type systems over unparameterised sorts [16].

A rejected alternative. A theory parameter can stand in for a type parameter — `Theory List(e: EmptySet)` is a list of whatever `e` exports. But then M is a map from theories to theories, the feature climbs back to the level of C and H, and the sharing discipline objects: `List(List(s))` reintroduces the list constructors along its own chain and draws `[repeat-label]`. The diagnostic is correct, and the alternative is rejected.

3.2 The `Monads` builder

A new postfix builder declares one or more data monads. Its entries name the monad, the sort it is a monad on, the interpretation of the operations, and the capabilities it claims. The following is the prelude declaration of the list monad, written with a sort variable `A` that Step 2 makes available to users and that the prelude is permitted to use from the start.

```
Monads {
  Seq over List {
    unit : (a) ~> [a];
    flatMap : ([], ^x.k) ~> [];
    flatMap : ([a, ...r], ^x.k) ~> (Concat (subst ^x.k a) (flatMap r ^x.k));
    zero : [];
    measure : Length;
    algebra : Proc by seq;
    laws : LeftUnit, RightUnit, Assoc;
  }
  Bag over HashBag {
    unit : (a) ~> {a};
    flatMap : ({}, ^x.k) ~> {};
    flatMap : ({a, ...r}, ^x.k) ~> (Union (subst ^x.k a) (flatMap r ^x.k));
    zero : {};
    commutative;
    measure : Size;
    algebra : Proc by PPar;
    laws : LeftUnit, RightUnit, Assoc, Comm;
  }
}
```

The surface is illustrative; its content is not. Five decisions are embedded in it.

The interpretation is a set of rewrites. The DDL defines operations only through rules, so unit and flatMap are given by cases over the sort's constructors, with the continuation as a first-class abstraction `^x.k` of sort `[A -> M(B)]`. These are ordinary rewrites: they are oriented, they fire,

and they "wiggle" like any other rule of the theory. map may be given explicitly for efficiency; when it is, $\mathsf{map}\ f = \mathsf{flatMap}(-, \mathsf{unit} \circ f)$ becomes one more obligation.

The laws are obligations, not equations. Matching in f1r3|@ng is matching modulo the equations of the theory that types the channel. Placing $\mathsf{flatMap}(m, \lambda x.\mathsf{unit}(x)) = m$ in an `Equations` block would put the matcher into higher-order unification, which is undecidable in general. The `laws` entry therefore *names* the laws; the elaborator records them as obligations, discharged by proof (as the cost monad's laws are, in Rocq and Lean) or by bounded exhaustive checking (as in the Sage harness), and never hands them to the matcher.

Obligation 3.1 (Monad laws)**.** For every declared data monad, the left unit, right unit and associativity laws hold up to the equations of the theory.

Obligation 3.2 (Confluence with the host theory)**.** The flatMap rewrites have no critical pair with the other rewrites of the theory, modulo its equations, that fails to join.

Obligation 3.2 is the one that makes `foreach` deterministic, and §4.7 explains why that is essential.

Capabilities are claims, and each one licenses a feature. The four optional entries are not decoration. Each corresponds to one feature of the comprehension, and the elaborator refuses the feature where the capability is absent.

Capability	Meaning	Licenses
`zero`	a unit for the monad's sum (MonadPlus)	a crisp `where` clause inside `foreach` (§4.4)
`commutative`	order of independent binds is immaterial [4]	the join `&` inside `foreach` (§4.3)
`measure`	the number of times $\mathsf{flatMap}(m, k)$ invokes k	precharge for a linear funding proof (§6.3)
`algebra`	a map $\mathsf{M}(\mathsf{Proc}) \to \mathsf{Proc}$	a `yield` whose body is a process (§4.5)

Each capability carries its own obligation: commutativity is a law; the measure must be exact (§6.3); the algebra must satisfy the Eilenberg–Moore laws [2].

One sort may carry several monads. The monads are named (`Seq`, `Bag`) because a sort can legitimately carry more than one monad structure, just as a rig carries two monoids on one carrier. A comprehension may name the monad it binds in, `foreach[Seq](...)`; the qualifier may be omitted when exactly one monad is declared on the source's sort.

3.3 Monads and the algebra of theories

A `Monads` entry is an element like any other, so it receives an origin token at introduction and is carried through inheritance.

- **Join.** Two theories that inherit `Seq` from one shared instance join cleanly, and the comprehension sees one monad. Two theories that independently declare a monad under the same name on the same sort draw `[join-collision]`: they are two structures with one spelling.

- **Difference.** Subtracting a constructor that a monad's rules mention leaves the declaration dangling. That is a new diagnostic, `[dangling-monad]`, and it is the analogue of the existing `[forward-reference]`.
- **Ordering.** A `Monads` block must follow the `Terms` blocks whose labels its rules mention, by the existing chain discipline.
- **Replacements.** Renaming a constructor preserves its identity and therefore preserves the monad. Replacing a constructor by one of different arity reproduces the open arity-drift problem already recorded for `PPar`: the inherited flatMap rules are written over a shape that no longer exists. Whatever resolution is adopted there — arity-preserving replacements, or a way to declare inherited rules structural — must be adopted here as well.
- **Parameters.** Once Step 2 lands, monad transformers are theories that take a monad as a parameter. This is the fourth rung of the DDL doing what it was built for.

4 The `foreach` comprehension

4.1 Syntax

```
Proc ::= ... | Foreach | Foreach ";" Proc
Foreach ::= "foreach" Qual? "(" Groups ")" "yield" Body
Qual ::= "[" MonadName "]"
Groups ::= Group ( ";" Group )*
Group ::= Bind ( "&" Bind )* ( "where" Cond )?
Bind ::= Pattern "<-" Term -- Term of a monadic sort
Body ::= "{" Proc "}"
```

The surface is chosen to coincide with `for` wherever the meanings coincide, and to diverge visibly where they do not. Its cost-accounted form is the subject of §6; until then, gates are elided.

4.2 Static discipline

Sources are terms of a monadic sort. In `for`, a source is a channel. In `foreach`, a source is a term whose sort carries a declared monad. A channel may not appear as a `foreach` source, because a receipt can block forever and would break synchrony; a monadic term may not appear as a `for` source except by being quoted into a name, in which case it is a channel and nothing more. The two comprehensions nest freely in each other's bodies.

This places a requirement on the CFL that it has not had before: the sort of the source expression must be known. Today f1r3|@ng patterns match untyped. `foreach` needs either sort inference for the source — from the pattern that bound it, from the theory that types the channel it arrived on — or an explicit qualifier.

One arrow. Only <− is meaningful. Persistence <= has no meaning over a value, and a peek «− is moot because a term source is never consumed. Both are refused rather than reinterpreted.

Typing. Writing Γ for the bindings in scope,

$$\frac{\Gamma \vdash m : \mathsf{M}(A) \qquad p : A \rightsquigarrow \Delta \qquad \Gamma, \Delta \vdash c\ \mathsf{cond} \qquad \Gamma, \Delta \vdash P : \mathsf{Proc}}{\Gamma \vdash \texttt{foreach[M](}\ p \leftarrow m\ \texttt{where}\ c\ \texttt{)}\ \texttt{yield}\ \texttt{\{}\ P\ \texttt{\}} : \mathsf{Proc}} \quad \begin{array}{l}\mathsf{M}\text{ has an algebra on }\mathsf{Proc}\\ c \neq \texttt{true} \Rightarrow \mathsf{M}\text{ has zero}\end{array}$$

with the evident extension to several groups (each group's Δ is in scope for every later group) and to `&` (which additionally requires M commutative).

4.3 Groups: `;` and `&`

Between groups, `;` is dependent bind: in

```
| foreach( x <- m ; y <- n(x) ) yield { P }
```

the source of the second group may mention `x`, and the comprehension means

$$\mathsf{flatMap}\big(m,\ \lambda x.\,\mathsf{flatMap}(n(x),\ \lambda y.\,\mathsf{unit}(P))\big).$$

The per-group scoping of `where` carries over unchanged.

Within a group, `&` means *independent* binds: neither source mentions the other's variables. Its meaning is order-independent only if the monad is commutative,

$$\mathsf{flatMap}\big(m,\ \lambda x.\,\mathsf{flatMap}(n,\ \lambda y.\,k)\big)\ =\ \mathsf{flatMap}\big(n,\ \lambda y.\,\mathsf{flatMap}(m,\ \lambda x.\,k)\big),$$

and so `&` is permitted only on monads that claim `commutative`: `Bag` and the finite powerset yes, `Seq` no. Without the restriction, `&` over a list would silently mean "left, then right," and the symbol would mean simultaneity in `for` and sequence in `foreach`.

4.4 `where` in `foreach`

A crisp `where` clause inside `foreach` is a filter, and a filter needs somewhere to send the candidates it rejects: the monad's zero. For a monad without `zero`, a `where` clause in `foreach` is an elaboration error, `[monad-no-zero]`.

The same keyword now has two operational readings, and the developer document should state both:

- in `for`, a failed guard *leaves the datum in the space*;
- in `foreach`, a failed guard *drops the binding*.

What reconciles them is that in both cases the rejected candidate is a non-event. Nothing is consumed, nothing is undone, and nothing downstream sees it.

4.5 `yield`, and the algebra on processes

The body of a `foreach` is a process, so the comprehension produces a value of sort $\mathsf{M}(\mathsf{Proc})$: a monadic collection of instantiated bodies. Something must turn that value into a running process. Categorically, that something is an Eilenberg–Moore algebra $\mathsf{alg} : \mathsf{M}(\mathsf{Proc}) \to \mathsf{Proc}$ [2].

- For `Bag`, the algebra is free. `PPar` *is* a `HashBag(Proc)`, so yielding over a bag produces a parallel composition, and the right-hand panel of Figure 1 is literal.
- For `Seq`, there are two algebras. One forgets the order and composes in parallel. The other composes in sequence, and exists only when the bodies are sequenceable (§5). The prelude declares the sequential one, and falls back to the parallel one with a warning when the bodies are not sequenceable.
- For a user-declared monad, a process-valued `yield` requires an `algebra` entry; without one the elaborator reports `[monad-no-algebra]`. A value-valued `yield`, producing an $\mathsf{M}(B)$ term rather than a process, does not need one, and is discussed with sequencing.

4.6 Reference semantics

The semantics of `foreach` is given by a translation into the core, which serves as the specification against which any implementation is judged (§7). For a single group over a built-in collection, the

translation walks the collection with a private recursive process. Each step consumes one element, matches it, evaluates the guard, and either adds the instantiated body to the accumulator or does not:

```
new walk, done in {
  walk!( m, Empty ) |
  contract walk( @rest, @acc ) = {
    match rest {
      [] => done!( acc )
      [a, ...r] => match a {
                     p where c => walk!( r, Add(acc, P) )
                     _ => walk!( r, acc )
                   }
    }
  } |
  for( @acc <- done ){ Alg(acc) }
}
```

`walk` and `done` are fresh, `Add` and `Empty` are the monad's sum and zero, and `Alg` releases the accumulated bodies via the algebra. For a user-declared monad the walk is replaced by the monad's own flatMap rules, with the continuation invoking the same match-and-guard step. Dependent groups nest the walk.

The translation was designed so that *each binding costs exactly one communication on a private name*: one send on `walk` per element. Entry and exit — the first send on `walk` and the send on `done` — cost two more, a constant per comprehension. The reference translation treats those two as administrative, funded by the cost-free unit in the way the internalisation retraction of the cost monad treats its administrative step [10], so its metered count is exactly one per binding. That choice is deliberate and is what makes per-binding charging (§6) coincide with the reference translation's own count.

4.7 Synchrony

"Synchronous" admits three readings, and only one is both meaningful and achievable.

Reading	Meaning	Assessment
Data dependency	bindings precede the body	too weak; `for` already has it
Completion barrier	a continuation waits for every body to finish	impossible in general; bodies spawn arbitrary processes
Atomic skeleton	the binding skeleton resolves without observable interleaving, then the bodies are released together	adopted

The atomic-skeleton reading has a precise formulation. In the reference translation, every communication of the skeleton is on a name bound by the translation's own `new`. Such communications are internal steps: nothing outside the comprehension can participate in them.

Proposition 4.1 (Synchrony)**.** *Suppose the internal reduction of a* `foreach` *skeleton is confluent and terminating. Then the translation of* `foreach( p <- m where c ) yield { P }` *is weakly bisimilar to a process that performs no visible action until it becomes* alg *applied to the instantiated bodies.*

Sketch. All skeleton steps are communications on names restricted by the translation, hence internal. Termination bounds the burst; confluence makes its end-point unique up to structural

congruence. A sequence of internal steps with a unique end-point is absorbed by weak bisimulation. □

So `foreach` is synchronous exactly when its skeleton is a deterministic, terminating burst of internal steps. The two hypotheses have different sources. Termination is supplied by cost accounting: a funded skeleton is a bounded one (§6). Confluence is Obligation 3.2, which is why that obligation is not optional. The slogan — concurrent internally, sequential externally — is the content of the proposition.

There is one interaction with the outside that the proposition does not cover: token consumption. A skeleton that draws its tokens one binding at a time from a purse shared with other processes is observable through that purse. §6.2 closes the gap.

5 Sequencing

Rholang 1.1's synchronous send sequences on an acknowledgement. `foreach` has a natural acknowledgement of its own — the end of the skeleton — so it can be sequenced the same way:

```
foreach( p <- m ) yield { P } ; K
  == new r in { [[ foreach( p <- m ) yield { P } ]]_r | for( _ <- r ){ K } }
```

where $[\![-]\!]_r$, written `[[-]]_r` above, is the reference translation extended to signal on `r` when the skeleton completes and the bodies have been released.

Two differences from `!?` are worth stating.

- `x!?(Q); K` depends on a convention: the receiver on `x` must treat its first argument as a return channel and reply. A receiver that does not leaves `K` parked. `foreach`'s acknowledgement is sent by the comprehension itself, so if the skeleton is funded and terminates, `K` runs. Sequencing after a `foreach` is more reliable than sequencing after a send.
- Mixed chains need no further machinery. The chain

  ```
  x!?(Q); foreach( y <- m ) yield { P }; z!?(R); K
  ```

  is nested `new`/`for` pairs, and `;` means "then" throughout: between receipt groups, after a synchronous send, and after a `foreach`.

The sequenceable fragment. The default acknowledgement fires on skeleton completion, because bodies may spawn arbitrary processes and their completion is not defined. But some bodies do have a completion event: those built only from synchronous sends, sequenced `foreach`s, and a final tail.

```
SeqBody ::= Tail | x "!?" "(" Proc ")" ";" SeqBody | Foreach ";" SeqBody
```

For a sequenceable body, completion can mean body completion. Over a non-commutative monad such as `Seq`, the body for element $i{+}1$ starts after the body for element i completes: a genuinely sequential fold. Over a commutative monad, bodies run in parallel and the acknowledgement fires on a counted join of their completions. The class is syntactic and checked at elaboration, so a developer never guesses which guarantee applies: `foreach` is sequential exactly when its body is sequenceable, and otherwise its guarantee is the skeleton.

The value form. The synchronous send discards its acknowledgement. If `foreach`'s acknowledgement instead carried the result of a value-valued `yield`, the continuation could bind it,

```
| foreach( x <- m where c ) yield f(x) ; (@ys) K
```

which gives the value form of the comprehension — Scala's `for/yield`, LINQ's `select` — without a separate construct. This note leaves the value form to a second tranche (Question 10.3), but the translation leaves room for it.

6 Cost

6.1 Two gates

The cost-accounted form of the comprehension is

```
| {% foreach( ptrn <- mndInst ) yield {% P %}[t] %}[s]
```

and its two gates mean different things.

The inner gate is ordinary. `{% P %}[t]` is core syntax with core meaning: each instantiated body runs once a `t`-token is available, and a body whose gate is unfunded parks, independently of every other body. The body slot is already a cost-accounted term, and needs no new semantics.

The outer gate is per binding. In the core, a gate funds one event, the communication. In `foreach`, the event the comprehension witnesses is each *binding*, so `[s]` funds one token per candidate. The principle is the one the developer document states — the witness is where cost attaches — with a different multiplicity, and the documentation should say so. Under the stack reading of budgets, the stack under `s` must be at least as deep as the number of candidates.

Per-group gates. The per-clause signed binds of the core carry over:

```
| {% foreach( {% x <- m %}[s1] ; {% y <- n(x) %}[s2] ) yield {% P %}[t] %}[s0]
```

funds the bindings of each group from its own lane. This matters for dependent groups, whose second-group candidates may be supplied by a different party from the first.

The syntax is forced. In the native four-sort grammar, the continuation slots of `for` and send have the signed-term sort $\mathbb{T}$. Giving `foreach` the same treatment — the comprehension in $\mathbb{T}$, the `yield` slot in $\mathbb{T}$ — produces exactly the surface above. An unwrapped `foreach` is not merely discouraged; it cannot be written in the cost-accounted surface, and wrapping by construction continues to hold. The same reasoning governs sequencing. The continuation after `;` is signed too, exactly as the continuation inside a desugared `!?` is:

```
| {% foreach( ptrn <- mndInst ) yield {% P %}[t] %}[s] ; {% K %}[u]
```

Proposition 6.1 (The outer cost does not depend on the filter)**.** *The number of tokens a `foreach` draws on its outer gate equals the number of candidates its skeleton evaluates, whether or not their guards hold. The number it draws on its inner gate is the number of survivors, and is bounded by the same count.*

Proof. Every candidate is matched and guarded before the skeleton decides whether to instantiate a body; the outer charge attaches to that evaluation. Only survivors instantiate bodies, and survivors are a subset of candidates. □

The proposition closes a hole that would otherwise exist. If only the inner gate charged, a `foreach` over a very large collection with a guard that is always false would cost nothing but a single token: the skeleton's work would be free. Per-binding charging prices the evaluation of every candidate, and Proposition 6.1 means a single estimate bounds both lanes.

6.2 The atomic outer gate

Proposition 4.1 leaves one interaction uncovered: drawing tokens one binding at a time from a shared purse is observable, and a skeleton that runs out mid-traversal has consumed something while producing nothing.

For a single group over a monad with a measure, the gap closes completely. The number of candidates is $n = \mathsf{measure}(m)$, computable when the gate opens, so the outer gate can be an n-ary join on the purse: one communication that takes n tokens or takes none.

- If the purse holds n tokens, the join fires, the skeleton runs as a burst of internal steps, and the comprehension is atomic in the strong sense: one external event.
- If it does not, the join does not fire, nothing is consumed, and the comprehension is a non-event — exactly as an unfunded communication is in the core.

There is no partial state and nothing burned. The design question this note earlier left open — reserve the whole depth up front, or burn tokens spent on an unfinished skeleton — is answered for this case by the n-ary gate.

Dependent groups reopen it. The number of second-group candidates, $\sum_x \mathsf{measure}(n(x))$, is known only after the first group has bound. A dependent `foreach` can take its gates group by group, in which case a later group may find its lane empty after earlier groups have paid; or it can take a declared upper bound at entry, in which case it overpays whenever the bound is loose. Question 10.1 records the choice.

6.3 Precharge and linear funding proofs

Per-binding charging moves the variability of `foreach`'s cost out of the semantics and into estimation. A linear funding proof for a deploy must show that every gate it will meet is funded, so for a `foreach` it needs an upper bound on the measure of the monadic instance: the depth of the budget the outer gate will draw, and, by Proposition 6.1, a bound on the number of bodies the inner gate will fund.

Remark 6.2 (Two uses of "precharge"). In the node, *precharge* also names the escrow mechanism — precharge, run, refund — that the cost-accounting work removed in favour of gate-funded execution. That mechanism is not being reintroduced. Here *precharge* means the bound that a static funding proof assumes, and nothing is escrowed: the n-ary gate takes exactly n tokens when it fires.

What is estimated. For a single group, the count is $\mathsf{measure}(m)$. For dependent groups, $\mathsf{measure}(m) + \sum_x \mathsf{measure}(n(x))$. Nesting a `foreach` inside the `yield` body multiplies: the body's own outer gate is funded from the inner lane, so the inner lane's bound is the number of survivors times the body's demand. In general the precharge is a bound, not a number.

Where bounds come from. In decreasing order of precision:

1. *A closed instance at preparation time.* The measure is computed and the bound is exact.

2. *An instance bound by an earlier receipt or match.* Theory types say nothing about size, and a pattern such as `[a, b, ...rest]` supplies only a lower bound. The guard on that binding supplies the upper bound:

```
for( @m <- ch where m.size() <= 64 ) {
  {% foreach( x <- m ) yield {% P %}[t] %}[s]
}
```

   Here the `where` clause does double duty: it is the developer's filter, and it is the premise the estimator reads to discharge the precharge obligation. The honest default is to require such a guard, or a declared bound, before a `foreach` over received data may appear in a deploy that must be proven funded.
3. *At the gate.* For any measured monad, the exact measure is available when the outer gate opens, and the n-ary gate of §6.2 uses it. This complements the static proof rather than replacing it: admission still needs the bound.

The measure obligation. For the built-in collections, the measure is size. For a user-declared monad, size is not intrinsic — for a tree, the relevant count may be the number of leaves — so the `measure` entry names it, and carries an obligation.

Obligation 6.3 (Exact measure)**.** For every instance m and every continuation k, the reduction of $\mathsf{flatMap}(m, k)$ invokes the continuation k exactly $\mathsf{measure}(m)$ times.

This is a linearity condition on the continuation, and it sits naturally beside the linear proofs it serves.

Two admissibility classes. Monads without a measure — lazy streams, state threads, continuations — remain safe under per-binding charging: they draw a token per binding and park when the purse is empty, which is a non-event. But they cannot be precharged, and so can never pass a linear funding proof. The split should be explicit:

- *precharge-admissible*: the monad declares a measure and the instance has a bound;
- *exhaustion-only*: everything else.

A deploy that must be proven funded may use only the first class; the elaborator reports `[monad-unmeasured]` otherwise.

Lanes. The `s` and `t` gates draw on different per-signature purses, so the proof splits by signature, and the node's disjoint per-signature lane pools already enforce the split. If `t` may mention variables bound by the pattern — `{% P %}[#x]` — then each element pays for its own processing, which is useful when the elements of a collection come from different signers; the inner lanes then become data-dependent, and the proof must quantify over the elements' signatures rather than over a fixed lane (Question 10.2).

7 Implementation

7.1 The translation is the specification, not the strategy

`foreach` is a conservative extension of the core.

Proposition 7.1 (Conservativity)**.** *A program that does not use `foreach` has the same reductions, the same costs and the same observable behaviour with the extension as without it.*

Sketch. Every new rule has a `foreach` constructor on its left-hand side, and no existing rule produces one. The `Monads` builder adds rules only to theories that declare it. The cost rule attaches only to the new gate. □

It follows that nothing obliges an implementation to compile `foreach` into `for`. The translation of §4.6 is retained as the reference semantics, and an implementation may evaluate `foreach` natively. The correctness criterion has to account for cost.

Obligation 7.2 (Graded refinement)**.** The native evaluation of every `foreach` is weakly bisimilar to its reference translation under the graded, signature-labelled transition relation.

Because the reference translation performs exactly one metered private communication per binding (§4.6), and the native charge is one token per binding, graded bisimilarity states both requirements at once: the same behaviour, and the same bill. The relation already exists in the node's Rocq development, as the graded transition system and its Hennessy–Milner adequacy results [20].

Conservativity itself still needs checking against the reducer, since the new constructor must not change how existing terms reduce or are billed; that is an ordinary compatibility review.

7.2 What native evaluation reuses

Evaluating the skeleton natively is a structural reduction over a term — the same class of work as the reducer's `match`. Over a built-in collection, `foreach` is an iterated match. The spatial matcher, the binders, and the `where` guards on match cases with fall-through, which are already implemented, supply the per-binding pattern-and-guard step. What is new is the iteration, the accumulation into the algebra, and the release.

7.3 What native evaluation must not inherit

The free-reduction default. The node's consensus ledger counts only communication events. Non-communication structural reductions — `new`, `match`, `if` — emit diagnostic events that are not charged. A natively evaluated skeleton, being match-class work, would by default cost nothing however large the instance, and with host work unmetered by design, that would be free unbounded work. The per-binding charge of §6 is therefore mandatory, and it is a charge on a language-level event, not on host work, so it remains consistent with the decision to cap rather than meter host work. A hard protocol cap on skeleton size sits beside it.

The static demand analysis. The funding analysis counts the communication nodes reachable in a normalised term and relies on receiving a term already desugared; it keeps an explicit seam for a front-end that delivers something else. A native `foreach` is that something else: it is not a communication node, and its contribution is data-dependent. The analysis gains a `foreach` rule — the bound of §6.3 on the outer lane, the bound times the body's demand on the inner lane.

7.4 What changes in the representation

The built-in collection methods (`nth`, `union`, `toList`, and so on) are first-order, and the method-call node cannot carry a binder. A native `foreach` therefore needs a new process node carrying binders, most naturally reusing the receipt's bind-and-pattern encoding. That is a change to the protobuf model, and so it is consensus-visible. It should be gated on the protocol version, as

the new cost-record fields are, and it must pass the compatibility review the frontend admission contract requires, since that contract freezes the model and protobuf files [20].

7.5 What gets simpler

- **Replay.** A native skeleton never touches the tuple space. There are no fresh names, no produce or communication events, and nothing in the event log. Replay only has to recompute, and recomputation is deterministic by Obligation 3.2; that obligation becomes the whole replay story.
- **Sequencing.** `foreach(...) yield {...}; K` releases `K` directly when the skeleton completes, without the two acknowledgement communications the synchronous send pays. Waiting on the acknowledgements of sequenceable bodies is still real communication; natively, it is a counted join on the node.
- **Order-then-execute.** The skeleton's internal steps race with nothing, across deploys or within one, so a `foreach` introduces no conflicts to mark by signature. Under the order-independent charge model this makes `foreach` unusually friendly to the concurrent executor: it is order-independent and replay-deterministic by construction, given confluence.

7.6 Built-in and declared monads

Two evaluation paths coexist.

- *Built-in collections* get a fast path in the reducer.
- *Monads declared in a theory* run their flatMap rules through the installed language's rewrite engine — the semantic-kernel service through which installed languages reduce terms. That work sits today under host-side semantic-work limits rather than the token ledger. The per-binding charge must cover this path too, or user-declared monads become the free route the built-ins were closed against.

7.7 Where the frontend stands

On the integration branch, `Module` and `Theory` declarations are parsed structurally, the existing lowerer emits the node's process representation directly, and public activation is off [20]. The work this note adds lands in three places: the elaborator (the `Monads` builder, its obligations and capability flags, and the new diagnostics), the sort checker (inferring source sorts), and the reducer and its funding analysis (the native node, its charge and its demand rule).

8 Interactions with the rest of f1r3|@ng

8.1 Channel types

A channel's type is an elaborated theory, so a `Monads` declaration travels with it. A receiver may `foreach` over a received term only when the channel's theory carries a monad on the term's sort. Nothing further is needed, and this is the right behaviour: two agents sharing a language definition share its iteration structure as well as its grammar.

8.2 Generated logics

The new constructors and rules are visible to the logic generated from the theory: unit and flatMap to the spatial connectives, their rewrites to the behavioural modality. That needs no special treatment.

There is a richer option. The logical counterpart of a monad is a *lax modality* [3, 1], written $\bigcirc\varphi$: "after binding in M, φ." A `Monads` declaration could generate $\bigcirc_{\mathsf{M}}$ in the generated logic, giving `where` clauses a way to speak about what a monadic value will deliver without enumerating it. The lax modality is a type-system-flavoured connective, and it raises no new adequacy question: it is consistent with the established position that Hypercube-generated logics are type systems and are not expected to be adequate.

8.3 Graded `where` clauses

Here the choose-versus-sum distinction of §1 does its work. A graded `where` clause takes values in a resolution algebra $\mathcal{V}$ rather than in the Booleans [12].

- In `for`, a graded guard *weights candidates in a race*: the weight changes which candidate is chosen.
- In `foreach`, there is no race. A graded guard must *scale contributions*: the weight multiplies what a candidate adds to the sum.

Scaling contributions requires M to be a semimodule over $\mathcal{V}$ — concretely, to be or embed in the finitely supported $\mathcal{V}$-weighted distribution monad. The declaration must therefore say which value algebras a monad supports, and a graded `where` on a `foreach` over a plain `Seq` is refused.

8.4 Simulation

A `foreach` over a weighted distribution monad is a stochastic simulation step: programming in the large, written with the same surface as programming in the small [17]. The "sum of all possible futures" of the equational route to a quantum reading of rho is what a `foreach` over a weighted bag computes.

8.5 The quantum fence

Kleisli composition in the vector-space monad is matrix multiplication, which is why the rho–linear-algebra correspondence and the comprehension meet. But a body that uses its bound variable twice is not linear, and the quantum reading of graded `where` clauses is restricted to the linear fragment of rho. The linearity check of that fragment applies to `foreach` bodies unchanged; a non-linear body is outside the quantum reading.

8.6 Summary

Feature	Interaction with `foreach`
Theory-level monads	laws hold modulo erasure in $\mathsf{C}(T)$, $\mathsf{H}(T)$ (Prop. 2.1)
Equations	laws are obligations, never equations; no higher-order matching
Rewrites	interpretation is rewrites; confluence obligation
Algebra of theories	origin tokens; `[join-collision]`, `[dangling-monad]`; arity drift shared with `PPar`
Arrows	only `<-`
`;` and `&`	dependent bind; `&` only on commutative monads
`where`	needs `zero`; drops rather than leaves
Synchronous send	`;` sequences `foreach`, more reliably than `!?`
Cost	two gates; per-binding outer charge; n-ary atomic gate; precharge by measure
Channel types	declaration travels with the theory
Generated logics	new constructors visible; optional lax modality
Graded `where`	scales contributions; needs a $\mathcal{V}$-semimodule
Quantum reading	linear bodies only
Order-then-execute	no conflicts; replay-deterministic given confluence

9 A first tranche

1. The prelude declares `Seq`, `Bag` and the finite powerset on the three built-in collections, with `zero`, `measure` and `algebra`, and with `commutative` on the latter two.
2. `foreach` with `<-`, `;` and crisp `where`; `&` only on commutative monads; a process-valued `yield`.
3. The cost-accounted surface with both gates, per-binding outer charge, and the n-ary outer gate for single-group comprehensions.
4. Sequencing with `;` after a `foreach`, acknowledging on skeleton completion; the sequenceable fragment.
5. Native evaluation in the reducer, the new process node gated on protocol version, and the demand rule in the funding analysis.
6. The laws, confluence and measure recorded as obligations, discharged for the prelude monads.

Deferred: sort parameters and user-declared monads (Step 2); the value form; graded `where` in `foreach`; the lax modality.

10 Open questions

Open question 10.1 (Dependent groups). For a `foreach` with dependent groups, should the outer gates be taken group by group, accepting that a later group may find its lane empty after earlier groups have paid, or should a declared upper bound be taken at entry, accepting overpayment when the bound is loose?

Open question 10.2 (Per-element funding). May the inner signature mention variables bound by the pattern, `{% P %}[#x]`? It lets data pay for its own processing, at the price of data-dependent lanes in the funding proof.

Open question 10.3 (The value form). Should the value form be introduced through the acknowledgement, `foreach(...) yield e ; (@ys) K`, or as a separate expression form?

Open question 10.4 (Arity drift). Whatever is decided for arity-changing replacements of `PPar` must be decided for monad interpretations. Which resolution is preferred?

Open question 10.5 (Default monad). Should a sort with exactly one declared monad allow the qualifier to be omitted, as proposed, or should the qualifier always be required for readability?

References

[1] P. N. Benton, G. M. Bierman, and V. C. V. de Paiva. Computational types from a logical perspective. *Journal of Functional Programming*, 8(2):177–193, 1998.

[2] S. Eilenberg and J. C. Moore. Adjoint functors and triples. *Illinois Journal of Mathematics*, 9(3):381–398, 1965.

[3] M. Fairtlough and M. Mendler. Propositional lax logic. *Information and Computation*, 137(1):1–33, 1997.

[4] A. Kock. Monads on symmetric monoidal closed categories. *Archiv der Mathematik*, 21:1–10, 1970.

[5] E. Meijer, B. Beckman, and G. Bierman. LINQ: reconciling object, relations and XML in the .NET framework. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, page 706, 2006.

[6] E. Moggi. Notions of computation and monads. *Information and Computation*, 93(1):55–92, 1991.

[7] P. Wadler. List comprehensions. In S. L. Peyton Jones, *The Implementation of Functional Programming Languages*, chapter 7. Prentice Hall, 1987.

[8] P. Wadler. Comprehending monads. *Mathematical Structures in Computer Science*, 2(4):461–493, 1992.

F1R3FLY.io working notes. Each lives in the F1R3FLY.io `publications` repository, in the directory given.

[9] L. G. Meredith. Cost-accounted rho calculus: a spectral decomposition of phlogiston. F1R3FLY.io working note, 2026. `cost-accounting/`.

[10] L. G. Meredith. Continued interactive GSLTs and the cost endofunctor. F1R3FLY.io working note, 2026. `cost-accounting-as-monad/`.

[11] L. G. Meredith. f1r3|@ng for the working software developer. F1R3FLY.io working note, 2026. `MeTTaIL4WorkingDev/f1r3lang4dev/`.

[12] L. G. Meredith. Graded where-clauses: non-crisp truth values as the syntax of non-determinism resolution in the rho calculus. F1R3FLY.io working note, 2026. `fuzzyware/`.

[13] L. G. Meredith. Graph-structured lambda theories: a reference for the working developer. F1R3FLY.io working note, 2026. `GSLT-intro/`.

[14] L. G. Meredith. History as an endofunctor: reversibility, covering spaces, and the memory price of energy conservation in cost-accounted GSLTs. F1R3FLY.io working note, 2026. `history-functor/`.

[15] L. G. Meredith. Two erasures and two laxities: internalising the cost and history monads over continued interactive GSLTs. F1R3FLY.io working note, 2026. `history-and-cost/`.

[16] M. Stay, L. G. Meredith, and J. B. Wells. Generating hypercubes of type systems. F1R3FLY.io working note, 2026. `drafts/Hypercube/`.

[17] L. G. Meredith. Weighted graph-structured lambda theories: stochastic and quantum execution for f1r3|@ng. F1R3FLY.io working note, second version, 2026. `rho-life/`.

[18] L. G. Meredith and M. Radestock. A reflective higher-order calculus. *Electronic Notes in Theoretical Computer Science*, 141(5):49–67, 2005.

Software.

[19] F1R3FLY.io. `f1r3node-rust`, branch `feature/module-syntax`: the `mettail-elab` reference elaborator for the `Module` and `Theory` surface. `https://github.com/F1R3FLY-io/f1r3node-rust`.

[20] F1R3FLY.io. `f1r3node-rust`, branch `feature/f1r3lang-mettail-only`: the frontend admission contract (`docs/design/f1r3lang-frontend.md`), the evaluation composition (`docs/node/f1r3lang-evaluation.md`), the cost-monad correspondence record and graded adequacy development (`docs/theory/`), the synchronous-send normaliser, and the funding analysis (`rholang/src/rust/interpreter/accounting/delta_sigma.rs`). `https://github.com/F1R3FLY-io/f1r3node-rust`.